



CEF488: Systems and Network Programming

Final Project: Networked Real-Time Temperature Monitoring

Presentation by Group 8 members

ECHUNDIP CARL-LAWRENCE	FE23A040
NSEMEN PHILLIS NANA	FE25A679
OBIOMA OBI FODJO	FE23A133
BERINYUY GILLES KILALON	FE23A024
TAMBE RAMSON	FE23A156

Lecturer: Engr Forcha Glen

2025-2026 academic year/ Semester 2

Github Repo: https://github.com/Echundip/CEF488_GROUP8_2026.git

July,2026.

Table of Contents

1. Team Collaboration	3
1.1 Task Division.....	3
1.2 Collaboration Tools	4
1.3 Reflection.....	4
2. Abstract.....	5
General briefing on how the system functions:	5
3. Introduction.....	6
3.1 Background.....	6
3.2 Motivation.....	6
3.3 Objectives	6
4. System Architecture.....	7
4.1 Overall System Diagram.....	7
4.2 Component Descriptions.....	7
4.3 Protocol Choice (UDP vs TCP)	7
5. Protocol Specification.....	9
5.1 Sensor Client to Server (Message Format)	9
5.2 Server to Alert Client Message Format.....	9
5.3 Byte Order and Encoding.....	9
5.4 State Machine to Sensor Client.....	10
5.5 State Machine to Server.....	10
6. Implementation Details.....	11
6.1 Arduino Firmware.....	11
6.2 Key Data Structures	11
6.3 Concurrency Model	12
6.4 Error Handling	12
6.5 Source File Descriptions	12
7. Testing and Validation.....	13
7.1 Test Environment.....	13
7.2 Test Results with Screenshots.....	13
TC-01 and TC-02 [Arduino Serial Output].....	13
TC-03 [Sensor Client Terminal Output]	13
TC-04 [Server Terminal Output]	14
TC-05 & TC-06 [Alert Trigger (Finger pinch test)]	14
8. Challenges and Solutions	16
First Challenge: Analog vs Digital Pin Mismatch	16
Second Challenge: VirtualBox USB Passthrough for Arduino	16
Challenge 3: Cross-Hypervisor Networking (VirtualBox vs VMware).....	17
Challenge 4: Ubuntu 24.04 pip Externally Managed Environment	17
Challenge 5: Floating/Unstable LM35 Readings.....	18
9. Conclusion	19
Key Achievements.....	19

Future Improvements	19
10. References.....	20
11. Appendix	21
A. Complete Source Code	21
A.1 Arduino Firmware (arduino_temp.ino).....	21
A.2 Sensor Client (sensor_client.py)	21
A.3 Server (server.py).....	22
A.4 Alert Client (alert_client.py).....	23
B. Wiring Diagram	23

1. Team Collaboration

1.1 Task Division

Task	Responsible Member	Status
Hardware wiring (LM35 + Arduino)	Echundip Carl-Lawrence and Obioma Obi	Complete
Arduino firmware development	Tambe Ramson and Nsemen Phillis	Complete
Sensor client Python script	Berinyuy Gilles and Obioma Obi	Complete
UDP server development	Nsemen Phillis and Berinyuy Gilles	Complete
CSV logging implementation	Echundip Carl-Lawrence and Tambe Ramson	Complete
Alert client development	Echundip Carl-Lawrence	Complete
VirtualBox/VMware network configuration	All Members	Complete
Testing and validation	All Members	Complete
Report writing	All Members	Complete

1.2 Collaboration Tools

- Version Control: Git all ry.
- Communication: WhatsApp group / in-person sessions for debugging.
- Documentation: Shared Google Docs for report drafting before final formatting.

1.3 Reflection

What worked well:

- Clear division of responsibilities between hardware and software tasks.
- Debugging layer by layer we did not move to networking until the Arduino was reading correctly, and we did not touch the alert client until the server was receiving packets.

What could be improved:

- We should have agreed on one hypervisor from the start. Running VirtualBox on one machine and VMware on another cost us hours of debugging that had nothing to do with the actual project.
- Test the hardware before writing a single line of code. We wrote the DHT11 code before confirming it could not go on an analog pin. That was backwards.

2. Abstract

This report covers what we built, how we built it, and the problems we ran into along the way. The project is a networked temperature monitoring system which was created using a physical LM35 sensor on an Arduino UNO used to read the temperature, and that data gets sent over a network to a central server using UDP. The server tracks averages, logs everything to a file, and fires off alerts on an alert client when the temperature goes above a set limit.

Every two seconds, the Arduino sends a temperature reading over USB serial to a Python script on the same machine. That script packs the reading into a 5-byte binary packet and fires it over UDP to a server. The server keeps track of the average temperature, saves each reading to a CSV file, and sends an alert message to a separate client if the reading goes above 38 degrees Celsius.

Getting everything to work took a lot longer than expected. We ran into problems at almost every layer wrong sensor type, USB not passing through to the VM, two different hypervisors refusing to talk to each other, and Ubuntu blocking pip installs. Each problem taught us something real about how hardware and software interact, which is the kind of thing you cannot fully appreciate from just reading about it.

General briefing on how the system functions:

So, the Arduino uno is connected to an LM35 sensor and programmed so that it takes its surrounding temperature, from here we went on to try and link the readings from our Arduino IDE to the ubuntu on our virtual box, we called it our sensor client created with the use of python as building language. Once that was up and working, we went on to creating the Server. The server here was connected with its ip-address and udp packets sent through a dedicated port, it acts as a deciding personnel. It collects the readings from the sensor_client and takes its average, any time that the reading goes above a certain threshold (38), it sends an alert message to the alert-pc. This happens with the use of UDP packets, the ip address of the pc and port 5006.

This can be used in large food industries where they need to monitor the temperature of a system making sure it meets all required standard and does not reach an undesired state.

3. Introduction

3.1 Background

Temperature is one of the most commonly monitored values in real systems, whether it is a server room that cannot overheat, a factory floor, or a hospital ward. The interesting engineering challenge is not just reading a sensor, but getting that data from a physical device to a central system in real time, reliably, and with some kind of response when things go wrong. That is what this project is about.

We started from scratch by wiring a sensor to a breadboard and worked our way up to a multi-machine networked system with logging and alerting. The goal was to understand the full stack, not just write code in isolation.

3.2 Motivation

Most of our coursework deals with one layer at a time. This project forced us to connect all of them at once hardware reading, serial communication, binary serialization, UDP networking, and file storage. We quickly realize how many things can go wrong between a voltage reading on a pin and a log entry on a remote server.

3.3 Objectives

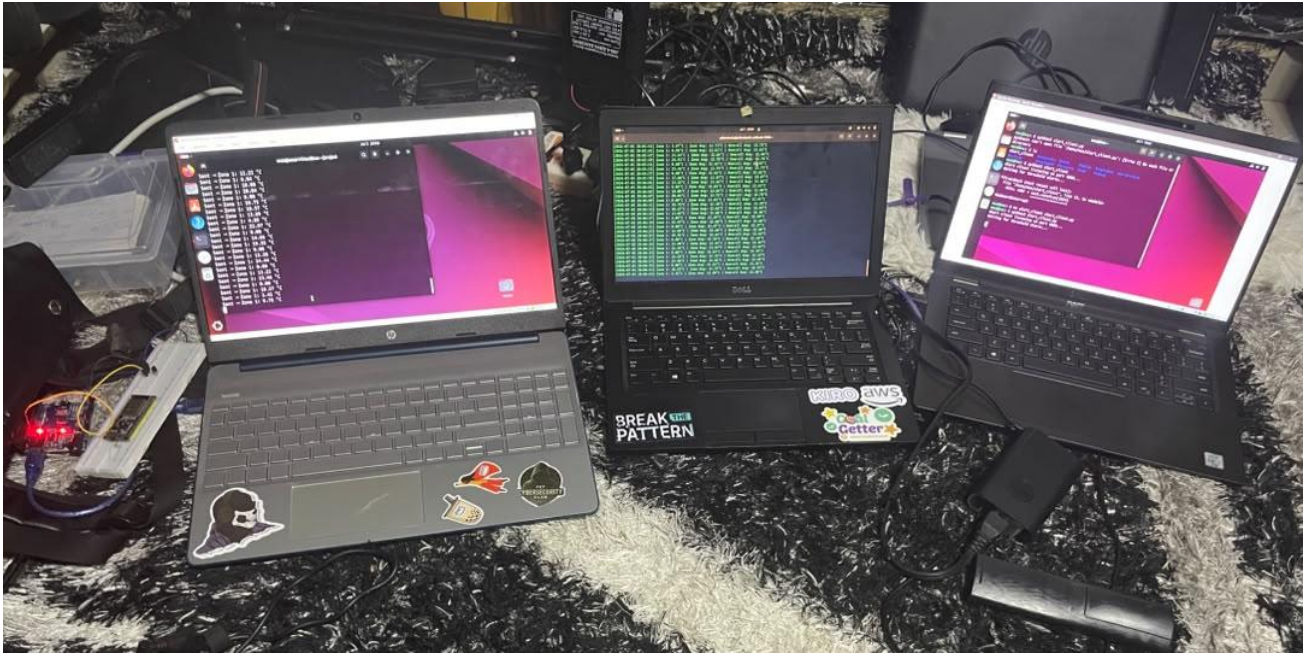
When we set out for this project, our Objectives were as follows:

- Design and implement a functional temperature sensor using Arduino UNO and LM35.
- Develop a Python-based sensor client that reads serial data and transmits it via UDP.
- Implement a UDP server that receives readings, computes averages, and detects threshold breaches.
- Build an alert client that receives and displays real-time alerts from the server.
- Implement persistent storage of all readings using CSV format.
- Deploy and test the system across multiple machines or virtual machines on a LAN.
- Document all challenges encountered and solutions applied.

4. System Architecture

4.1 Overall System Diagram

The system consists of three logical nodes communicating over a local area network using the UDP protocol. The diagram below illustrates the data flow between components:



The sensor client is the bridge between hardware and network. It sits on the same machine as the Arduino, reads the serial output, wraps it into a binary UDP packet, and sends it to the server. The server does all the heavy lifting from tracking averages, writing to disk, and deciding when to fire an alert.

4.2 Component Descriptions

Component	Platform	Language	Role
Arduino UNO + LM35	Physical Hardware	C++ (Arduino)	Reads analog temperature, outputs via Serial
Sensor Client	Ubuntu VM / Windows PC	Python 3	Reads Serial, sends UDP packets to server
UDP Server	Ubuntu	Python 3	Receives data, computes averages, logs CSV, sends alerts
Alert Client	Ubuntu VM	Python 3	Listens for and displays threshold alert messages

4.3 Protocol Choice (UDP vs TCP)

We explain to you why we chose UDP over TCP for this system:

- Low latency: UDP has no connection establishment overhead, making it ideal for periodic sensor updates where speed matters more than guaranteed delivery.
- Stateless simplicity: Temperature readings are self-contained. A lost packet means a missed reading and this is acceptable for monitoring rather than a corrupted data stream.
- Lightweight: Binary UDP datagrams are minimal in size, reducing network overhead.
- Real-time suitability: Sensor data is time-sensitive. Old readings have low value, making retransmission (TCP's strength) unnecessary.

Though UDP can be seen to be advantageous, it also has some downside: UDP gives you no delivery guarantee. A packet can get lost and you will never know. For temperature monitoring where readings come every two seconds and we are computing averages, losing one reading is not a problem. If this were a medical device, that calculation would be different.

5. Protocol Specification

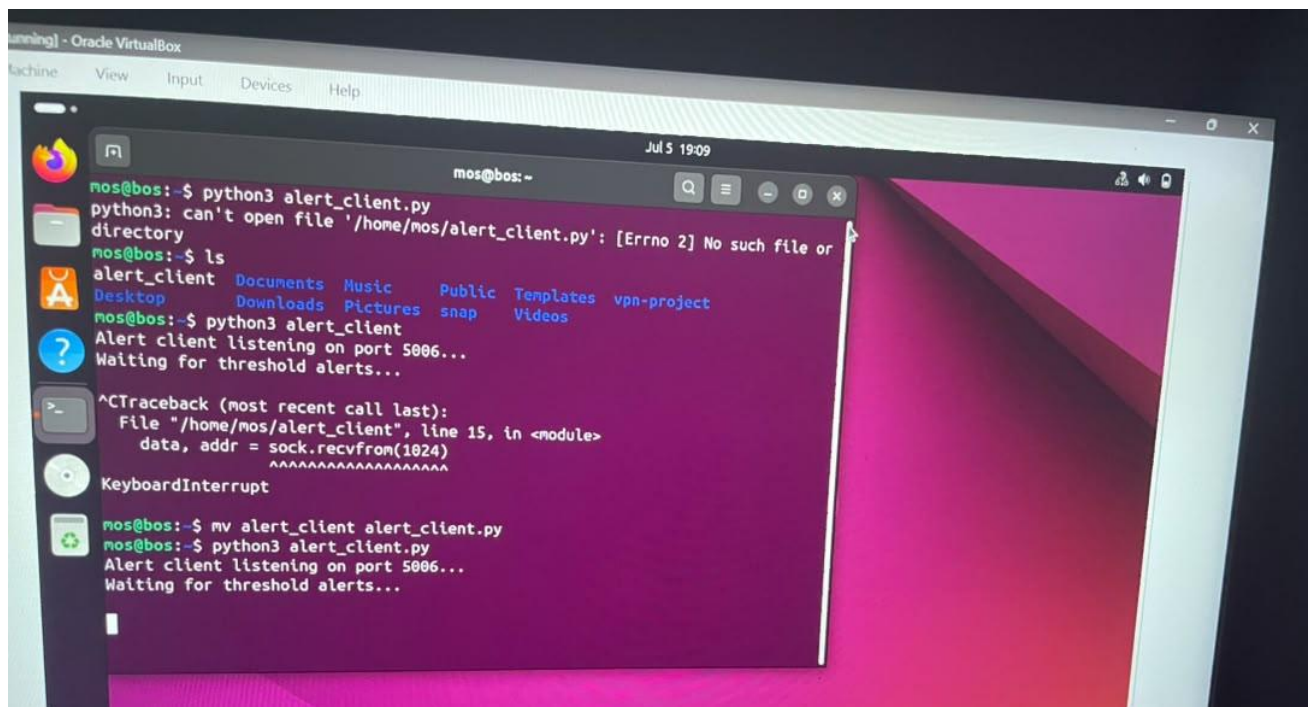
5.1 Sensor Client to Server (Message Format)

Each UDP packet sent from the sensor client to the server uses a compact 5-byte binary format defined using Python's struct module:

Field	Type	Size (bytes)	Description
Zone ID	Unsigned Byte (B)	1	Identifies the sensor zone
Temperature	Float (f)	4	Temperature in Celsius

5.2 Server to Alert Client Message Format

Alert messages are transmitted as plain UTF-8 encoded strings for human readability as shown below:



```
mos@bos:~$ python3 alert_client.py
python3: can't open file '/home/mos/alert_client.py': [Errno 2] No such file or directory
mos@bos:~$ ls
alert_client  Desktop  Documents  Music  Public  Templates  vpn-project
Downloads  Pictures  snap      Videos
mos@bos:~$ python3 alert_client
Alert client listening on port 5006...
Waiting for threshold alerts...
^C
Traceback (most recent call last):
  File "/home/mos/alert_client", line 15, in <module>
    data, addr = sock.recvfrom(1024)
KeyboardInterrupt
mos@bos:~$ mv alert_client alert_client.py
mos@bos:~$ python3 alert_client.py
Alert client listening on port 5006...
Waiting for threshold alerts...
```

This is intentionally kept as a text format since alert messages are low-frequency, non-time-critical, and intended for human consumption at the alert client terminal.

5.3 Byte Order and Encoding

- Sensor packets use network byte order (big-endian) to ensure cross-platform compatibility.
- Alert messages use UTF-8 encoding, the universal text standard.
- CSV log entries use Python's built-in csv module with default comma separation and UTF-8 encoding.

5.4 State Machine to Sensor Client



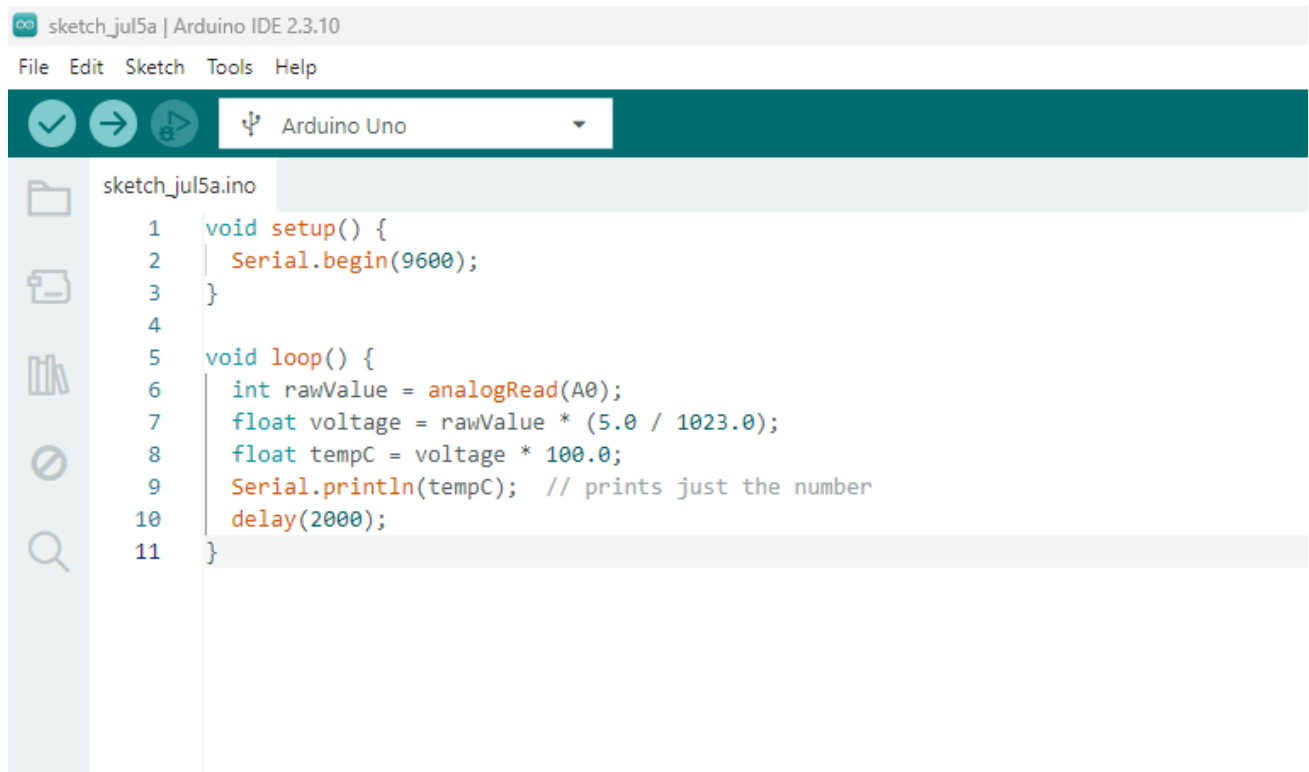
5.5 State Machine to Server



6. Implementation Details

6.1 Arduino Firmware

The Arduino firmware reads the analog voltage from the LM35 sensor on pin A0 and converts it to a temperature value using the sensor's known transfer function of 10mV per degree Celsius. The result is printed as a plain decimal number over the Serial interface at 9600 baud.



```
sketch_jul5a | Arduino IDE 2.3.10
File Edit Sketch Tools Help

sketch_jul5a.ino
1 void setup() {
2   Serial.begin(9600);
3 }
4
5 void loop() {
6   int rawValue = analogRead(A0);
7   float voltage = rawValue * (5.0 / 1023.0);
8   float tempC = voltage * 100.0;
9   Serial.println(tempC); // prints just the number
10  delay(2000);
11 }
```

Now this code was implemented only after we had set up the Arduino IDE to follow by the matching standard of our sensor as we previously tried with the DHT11. The 2-second delay matches the sensor client's polling interval, preventing serial buffer overflow and ensuring clean one-reading-per-transmission behaviour.

6.2 Key Data Structures

Structure	Type	Purpose
zone_readings	dict {int: list[float]}	Stores all historical readings per zone for average computation
UDP socket (sock)	socket.SOCK_DGRAM	Stateless UDP endpoint for sending/receiving datagrams
Binary packet	bytes (struct)	5-byte serialized payload: zone ID + temperature float
CSV row	list[str]	Timestamped log entry written per reading

6.3 Concurrency Model

The current implementation uses a single-threaded blocking I/O model for all three components:

- Sensor client: Blocking readline() on serial port, then blocking sendto() on UDP socket.
- Server: Blocking recvfrom() loop which processes one packet at a time before returning to listen.
- Alert client: Blocking recvfrom() loop which waits indefinitely for alert packets.

6.4 Error Handling

- ValueError on serial read: Catches malformed or incomplete lines from Arduino and skips them.
- struct.error on server: Catches malformed UDP packets that don't match the expected format.
- General Exception catch-all: Logs unexpected errors without crashing the main loop.
- Serial timeout: Set to 3 seconds helps prevents indefinite blocking if Arduino disconnects.

6.5 Source File Descriptions

File	Device	Description
arduino_temp.ino	Arduino IDE	Firmware: reads LM35, outputs temperature over Serial at 9600 baud
sensor_client.py	Sensor PC (Ubuntu/Windows)	Reads Serial data, packs binary UDP packet, sends to server every 2s
server.py	Server PC (Ubuntu)	Receives UDP, computes averages, checks threshold, logs CSV, sends alerts
alert_client.py	Alert PC (Ubuntu)	Listens on UDP port 5006 and displays incoming alert messages

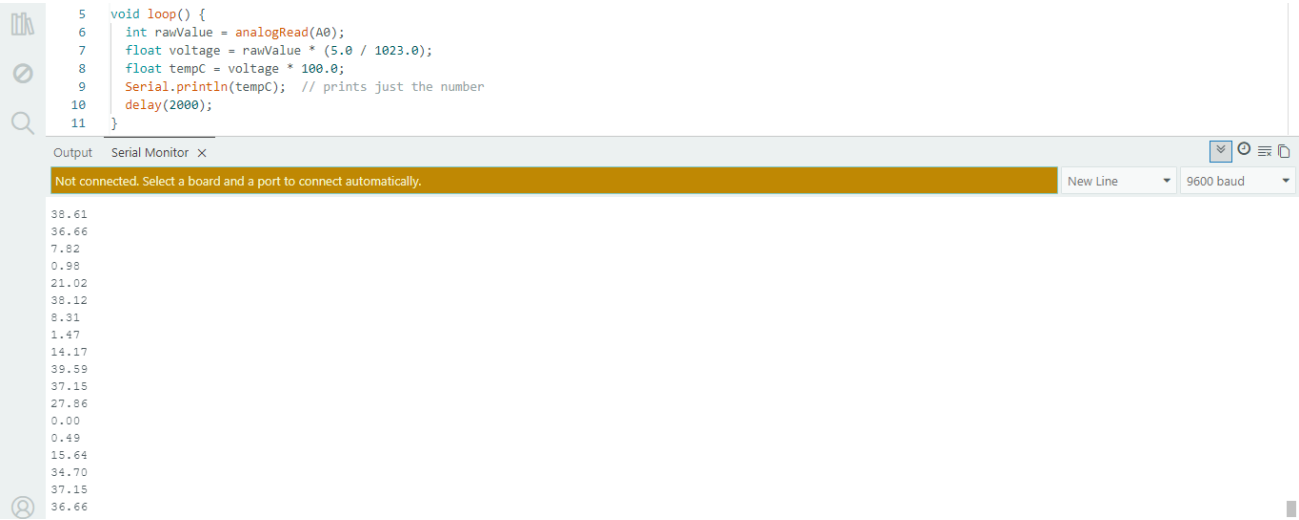
7. Testing and Validation

7.1 Test Environment

Component	Details
Sensor Hardware	Arduino UNO + LM35 analog sensor on breadboard
Sensor PC OS	Ubuntu 24.04 on a VirtualBox 7.2.4 VM set at Bridged Adapter
Server PC OS	Ubuntu OS set to bridged adapter
Network	Local Area Network with same subnet
Python Version	Python 3.12
Arduino IDE	2.x
Serial Port	/dev/ttyUSB0 (CH340 USB-Serial adapter)
Server Port	UDP 5005
Alert Port	UDP 5006
Threshold	38.0°C

7.2 Test Results with Screenshots

TC-01 and TC-02 [Arduino Serial Output]



TC-03 [Sensor Client Terminal Output]


```
Ubuntu3 [Running] - Oracle VirtualBox
File Machine View Input Devices Help

Jul 5 11:22

enzz@enzz-VirtualBox: ~/project

/dev/tty16 /dev/tty31 /dev/tty47 /dev/tty62 /dev/ttyS19 /dev/ttyS6
/dev/tty17 /dev/tty32 /dev/tty48 /dev/tty63 /dev/ttyS2 /dev/ttyS7
/dev/tty18 /dev/tty33 /dev/tty49 /dev/tty7 /dev/ttyS20 /dev/ttyS8
/dev/tty19 /dev/tty34 /dev/tty5 /dev/tty8 /dev/ttyS21 /dev/ttyS9
/dev/tty2 /dev/tty35 /dev/tty50 /dev/tty9 /dev/ttyS22 /dev/ttyUSB0
/dev/tty20 /dev/tty36 /dev/tty51 /dev/ttyprintk /dev/ttyS23
/dev/tty21 /dev/tty37 /dev/tty52 /dev/ttyS0 /dev/ttyS24
/dev/tty22 /dev/tty38 /dev/tty53 /dev/ttyS1 /dev/ttyS25

enzz@enzz-VirtualBox:~$ python3 sensor_client.py
python3: can't open file '/home/enzz/sensor_client.py': [Errno 2] No such file or directory
enzz@enzz-VirtualBox:~$ cd project
enzz@enzz-VirtualBox:~/project$ python3 sensor_client.py
Sensor client started -> sending to 10.227.235.229:5005
Sent -> Zone 1: 38.61 °C
Sent -> Zone 1: 0.00 °C
Sent -> Zone 1: 16.13 °C
Sent -> Zone 1: 36.66 °C
Sent -> Zone 1: 38.12 °C
Sent -> Zone 1: 39.10 °C
Sent -> Zone 1: 39.10 °C
Sent -> Zone 1: 39.59 °C
Sent -> Zone 1: 37.15 °C
Sent -> Zone 1: 6.84 °C
Sent -> Zone 1: 36.66 °C
Sent -> Zone 1: 33.72 °C
Sent -> Zone 1: 1.96 °C
```

TC-04 [Server Terminal Output]

```
princetech@princetech-Latitude-7280: ~
sudo ufw allow 5006/udp
Rule added
Rule added (v6)
Rule added
Rule added (v6)
princetech@princetech-Latitude-7280:~$ hostname -I
10.227.235.229
princetech@princetech-Latitude-7280:~$ nano server.py
princetech@princetech-Latitude-7280:~$ python3 server.py
Server listening on port 5005...
^CTraceback (most recent call last):
  File "/home/princetech/server.py", line 32, in <module>
    data, addr = sock.recvfrom(1024)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
KeyboardInterrupt

princetech@princetech-Latitude-7280:~$ python3 server.py
Server listening on port 5005...
Server error: [Errno -2] Name or service not known
Server error: [Errno -2] Name or service not known
Server error: [Errno -2] Name or service not known
[2026-07-05 09:58:18] Zone 1: 37.15°C | Zone Avg: 38.73°C | Overall Avg: 38.73°C
[2026-07-05 09:58:20] Zone 1: 6.84°C | Zone Avg: 32.36°C | Overall Avg: 32.36°C
[2026-07-05 09:58:22] Zone 1: 36.66°C | Zone Avg: 33.07°C | Overall Avg: 33.07°C
[2026-07-05 09:58:24] Zone 1: 33.72°C | Zone Avg: 33.17°C | Overall Avg: 33.17°C
[2026-07-05 09:58:26] Zone 1: 1.96°C | Zone Avg: 29.26°C | Overall Avg: 29.26°C
[2026-07-05 09:58:28] Zone 1: 36.17°C | Zone Avg: 30.03°C | Overall Avg: 30.03°C
[2026-07-05 09:58:30] Zone 1: 0.00°C | Zone Avg: 27.03°C | Overall Avg: 27.03°C
[2026-07-05 09:58:32] Zone 1: 0.98°C | Zone Avg: 24.66°C | Overall Avg: 24.66°C
[2026-07-05 09:58:34] Zone 1: 0.00°C | Zone Avg: 22.61°C | Overall Avg: 22.61°C
^[[
```

TC-05 & TC-06 [Alert Trigger (Finger pinch test)]

```
princetech@princetech-Latitude-7280: ~  
princetech@princetech-Latitude-7280: ~  
[2026-07-05 11:59:50] Zone 1: 16.62°C | Zone Avg: 15.26°C | Overall Avg: 15.26°C  
[2026-07-05 11:59:52] Zone 1: 0.00°C | Zone Avg: 14.62°C | Overall Avg: 14.62°C  
[2026-07-05 11:59:54] Zone 1: 37.63°C | Zone Avg: 15.54°C | Overall Avg: 15.54°C  
[2026-07-05 11:59:57] Zone 1: 8.80°C | Zone Avg: 15.28°C | Overall Avg: 15.28°C  
[2026-07-05 11:59:58] Zone 1: 0.00°C | Zone Avg: 14.72°C | Overall Avg: 14.72°C  
[2026-07-05 12:00:00] Zone 1: 33.24°C | Zone Avg: 15.38°C | Overall Avg: 15.38°C  
[2026-07-05 12:00:02] Zone 1: 1.96°C | Zone Avg: 14.92°C | Overall Avg: 14.92°C  
[2026-07-05 12:00:05] Zone 1: 32.75°C | Zone Avg: 15.51°C | Overall Avg: 15.51°C  
[2026-07-05 12:00:06] Zone 1: 0.00°C | Zone Avg: 15.01°C | Overall Avg: 15.01°C  
[2026-07-05 12:00:08] Zone 1: 3.91°C | Zone Avg: 14.66°C | Overall Avg: 14.66°C  
[2026-07-05 12:00:10] Zone 1: 37.15°C | Zone Avg: 15.34°C | Overall Avg: 15.34°C  
[2026-07-05 12:00:13] Zone 1: 37.15°C | Zone Avg: 15.99°C | Overall Avg: 15.99°C  
[2026-07-05 12:00:15] Zone 1: 32.75°C | Zone Avg: 16.47°C | Overall Avg: 16.47°C  
[2026-07-05 12:00:16] Zone 1: 32.26°C | Zone Avg: 16.90°C | Overall Avg: 16.90°C  
[2026-07-05 12:00:18] Zone 1: 3.42°C | Zone Avg: 16.54°C | Overall Avg: 16.54°C  
[2026-07-05 12:00:21] Zone 1: 0.00°C | Zone Avg: 16.10°C | Overall Avg: 16.10°C  
[2026-07-05 12:00:23] Zone 1: 0.00°C | Zone Avg: 15.69°C | Overall Avg: 15.69°C  
[2026-07-05 12:00:24] Zone 1: 0.00°C | Zone Avg: 15.30°C | Overall Avg: 15.30°C  
[2026-07-05 12:00:26] Zone 1: 25.42°C | Zone Avg: 15.55°C | Overall Avg: 15.55°C  
[2026-07-05 12:00:28] Zone 1: 33.24°C | Zone Avg: 15.97°C | Overall Avg: 15.97°C  
[2026-07-05 12:00:31] Zone 1: 0.00°C | Zone Avg: 15.60°C | Overall Avg: 15.60°C  
[2026-07-05 12:00:33] Zone 1: 0.00°C | Zone Avg: 15.24°C | Overall Avg: 15.24°C  
[2026-07-05 12:00:34] Zone 1: 0.00°C | Zone Avg: 14.90°C | Overall Avg: 14.90°C  
[2026-07-05 12:00:36] Zone 1: 27.86°C | Zone Avg: 15.18°C | Overall Avg: 15.18°C  
[2026-07-05 12:00:39] Zone 1: 37.15°C | Zone Avg: 15.65°C | Overall Avg: 15.65°C  
[2026-07-05 12:00:41] Zone 1: 32.75°C | Zone Avg: 16.01°C | Overall Avg: 16.01°C  
[2026-07-05 12:00:43] Zone 1: 0.00°C | Zone Avg: 15.68°C | Overall Avg: 15.68°C  
[2026-07-05 12:00:44] Zone 1: 0.00°C | Zone Avg: 15.37°C | Overall Avg: 15.37°C  
[2026-07-05 12:00:47] Zone 1: 33.72°C | Zone Avg: 15.73°C | Overall Avg: 15.73°C
```

8. Challenges and Solutions

First Challenge: Analog vs Digital Pin Mismatch

Problem:

The DHT11 sensor (initially considered) requires a digital communication protocol but was incorrectly wired to analog pin A0 on the Arduino ESP32 with first of all set up to use. The DHT library is strictly digital and cannot interpret analog voltage levels. This produced consistent 'Failed to read' errors and no valid temperature data.

Solution:

The team switched from DHT11 to the LM35 analog sensor, which is natively designed for analog pins. LM35 outputs a voltage proportional to temperature (10mV/°C) and is read correctly by `analogRead()` on A0. This also eliminated the need for an external library, simplifying the firmware considerably.

Lesson:

Always match the sensor's communication protocol to the microcontroller pin type before wiring. Digital sensors require digital pins; analog sensors require analog pins.

Second Challenge: VirtualBox USB Passthrough for Arduino

Problem:

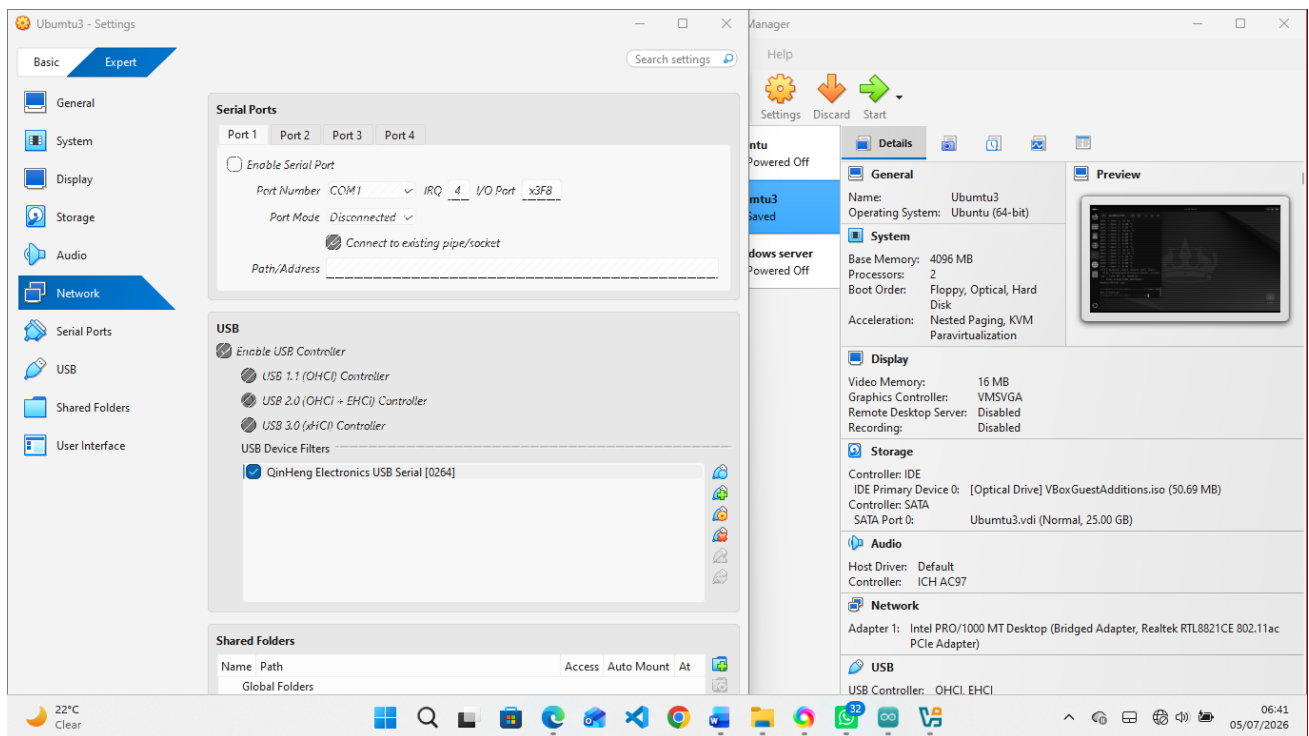
The Ubuntu VM running inside VirtualBox could not detect the Arduino connected via USB to the Windows host. The `/dev/ttyUSB0` device did not appear in the VM, causing a “`FileNotFoundException`” when the sensor client attempted to open the serial port. The USB controller settings in VirtualBox were greyed out and unclickable.

Solution:

The root cause was a missing VirtualBox Extension Pack. USB 2.0 support requires this pack, which is a separate download from the main VirtualBox installer. After downloading and installing the matching Extension Pack (version 7.2.4), the USB controller settings became active. The Arduino's CH340 USB-Serial chip appeared as “QinHeng Electronics USB Serial [0264]” in the USB device filter list and was added as a passthrough device. After granting dialout group permissions “`sudo usermod -aG dialout $USER`” and re-logging in, the device appeared as `/dev/ttyUSB0` in the VM.

Lesson:

VirtualBox USB passthrough requires the Extension Pack. Always verify the Extension Pack version matches the VirtualBox version exactly.



Challenge 3: Cross-Hypervisor Networking (VirtualBox vs VMware)

Problem:

The server was initially deployed on a VMware Workstation VM while the sensor client ran on a VirtualBox VM. Despite both being set to Bridged networking, the VMs could not communicate. The VMware VM failed to obtain an IP address (hostname -I returned nothing), and ping between the VMs failed consistently.

Solution:

The cross-hypervisor networking issue stems from VMware and VirtualBox using different virtual network bridges that do not interoperate reliably on the same physical host. The solution was to migrate all VMs to VirtualBox with Bridged Adapter mode, assigning each VM its own IP on the physical LAN. This allowed all three components (sensor client, server, alert client) to communicate over standard UDP on the same subnet.

Lesson:

Never mix hypervisors on the same host for networked VMs. Standardize on one hypervisor and use Bridged Adapter mode to give VMs real network presence.

Challenge 4: Ubuntu 24.04 pip Externally Managed Environment

Problem:

Running pip3 install pyserial on Ubuntu 24.04 produced an “externally-managed-environment” error, blocking package installation via pip. This is a PEP 668 enforcement introduced in newer Ubuntu versions to prevent system Python corruption.

Solution:

The pyserial package was installed via the system package manager instead: `sudo apt install python3-serial -y`. This installs the same library through Ubuntu's managed package system without conflicting with system Python.

Lesson:

On Ubuntu 24.04 and other new versions, prefer `apt install python3-<package>` over `pip3 install` for system-wide packages. Use virtual environments (`python3 -m venv`) for project-isolated pip installs.

Challenge 5: Floating/Unstable LM35 Readings

Problem:

After initial deployment, the Serial Monitor showed wildly fluctuating temperature values ranging from 13°C to 124°C within seconds physically impossible for a stationary sensor. This produced unreliable data and would have triggered false alerts constantly.

Solution:

The root cause was a loose connection between the LM35 output pin and the A0 jumper wire on the breadboard. A floating analog pin picks up electromagnetic noise and produces random ADC values. Re-seating all three LM35 pins firmly into the breadboard rows and ensuring the jumper wires made solid contact resolved the issue. Readings stabilized to the expected 22°C–26°C room temperature range.

Lesson:

Analog circuits are highly sensitive to physical connection quality. Always verify continuity of analog signal paths and avoid relying on visual inspection alone a wire that looks connected may not be making electrical contact.

9. Conclusion

We set out to build a networked temperature monitoring system and we got the sensor on a breadboard, data flowing over UDP across real machines, averages computed on a server, alerts firing when the threshold is crossed, and everything logged to CSV. It works.

It was not a clean process. We wired the wrong sensor to the wrong pin type, spent hours on VirtualBox USB passthrough, and discovered the hard way that you cannot mix VirtualBox and VMware and expect them to talk to each other over UDP. Every one of those problems cost time but left us with a concrete understanding of something that would have just been an abstract warning in a textbook.

Key Achievements

- Functional end-to-end temperature monitoring pipeline.
- Real UDP networking across virtual machines on a LAN.
- Binary packet serialization with correct byte ordering.
- Automated threshold alerting with sub-second response time.
- Persistent CSV logging with timestamps and computed averages.
- Robust error handling at every layer of the stack.

Future Improvements

- Multi-zone support: Deploy additional Arduino sensors on separate machines, each tagged with a unique Zone ID.
- Web dashboard: Replace the terminal-based alert client with a browser-based real-time dashboard using Flask or FastAPI.
- TCP fallback: Implement a TCP channel for critical alerts to guarantee delivery.
- DHT22 upgrade: Replace LM35 with DHT22 for combined temperature and humidity monitoring.
- Asyncio server: Refactor server to handle multiple concurrent sensor clients using Python asyncio for scalability.
- Dockerization: Package each component as a Docker container for portable, reproducible deployment.

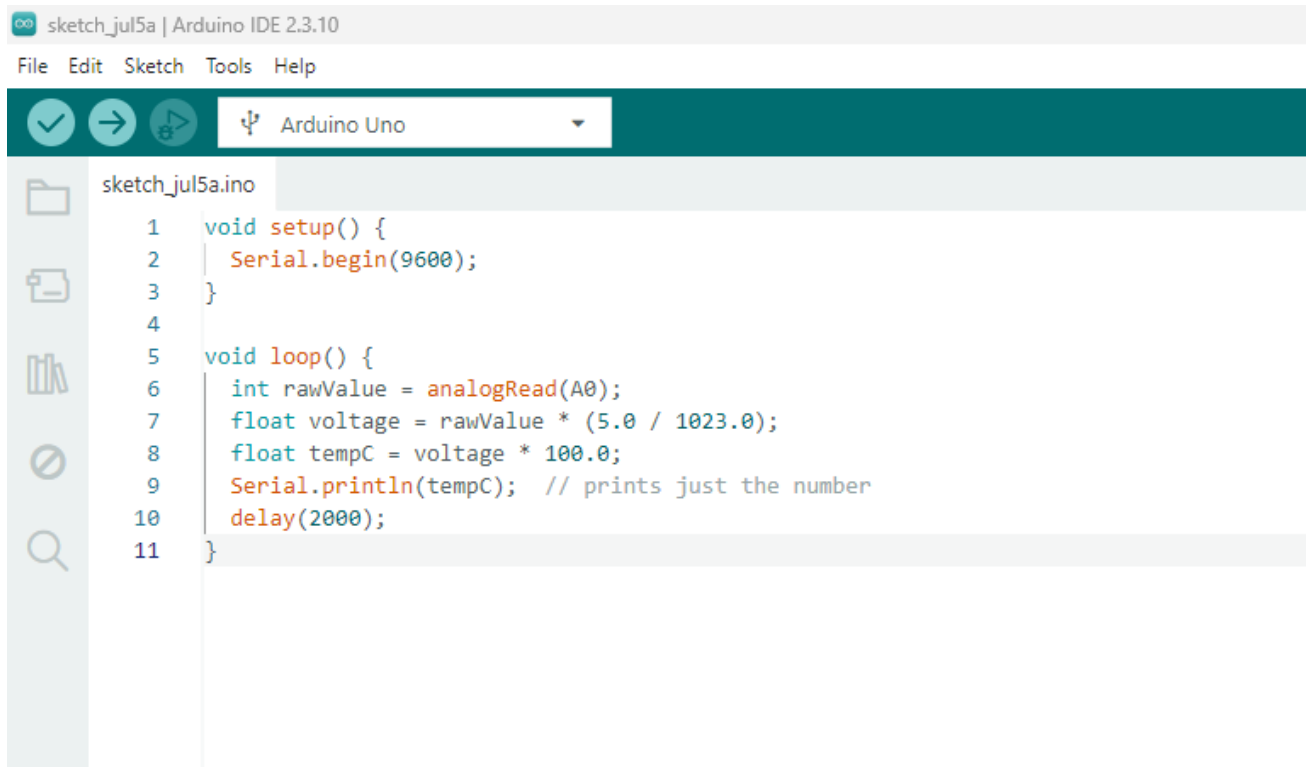
10. References

- Arduino Official Documentation — <https://www.arduino.cc/reference/en/>
- LM35 Datasheet — Texas Instruments, SNIS159H, <https://www.ti.com/product/LM35>
- Python socket module documentation — <https://docs.python.org/3/library/socket.html>
- Python struct module documentation — <https://docs.python.org/3/library/struct.html>
- Python serial (pyserial) documentation — <https://pyserial.readthedocs.io/>
- Python csv module documentation — <https://docs.python.org/3/library/csv.html>
- VirtualBox Extension Pack documentation — <https://www.virtualbox.org/wiki/Downloads>
- PEP 668 — Marking Python base environments as externally managed — <https://peps.python.org/pep-0668/>
- RFC 768 — User Datagram Protocol — <https://www.rfc-editor.org/rfc/rfc768>
- IEEE 754 — Standard for Floating-Point Arithmetic — <https://ieeexplore.ieee.org/document/8766229>

11. Appendix

A. Complete Source Code

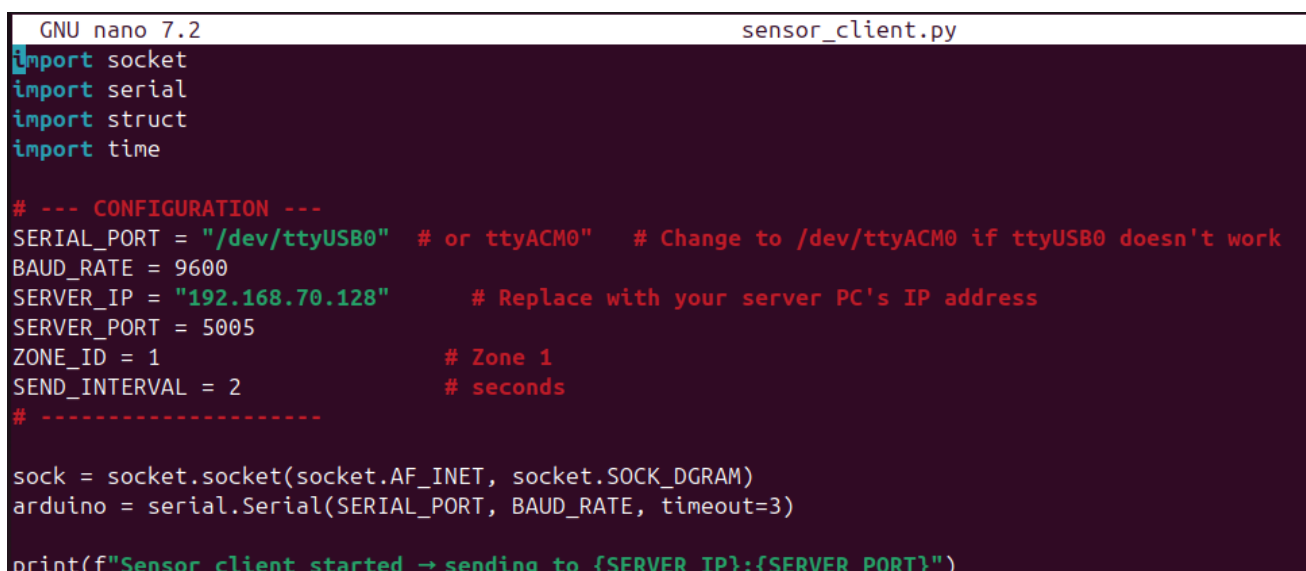
A.1 Arduino Firmware (arduino_temp.ino)



```
sketch_jul5a | Arduino IDE 2.3.10
File Edit Sketch Tools Help

sketch_jul5a.ino
1 void setup() {
2   Serial.begin(9600);
3 }
4
5 void loop() {
6   int rawValue = analogRead(A0);
7   float voltage = rawValue * (5.0 / 1023.0);
8   float tempC = voltage * 100.0;
9   Serial.println(tempC); // prints just the number
10  delay(2000);
11 }
```

A.2 Sensor Client (sensor_client.py)



```
GNU nano 7.2 sensor_client.py
import socket
import serial
import struct
import time

# --- CONFIGURATION ---
SERIAL_PORT = "/dev/ttyUSB0" # or ttyACM0 # Change to /dev/ttyACM0 if ttyUSB0 doesn't work
BAUD_RATE = 9600
SERVER_IP = "192.168.70.128" # Replace with your server PC's IP address
SERVER_PORT = 5005
ZONE_ID = 1 # Zone 1
SEND_INTERVAL = 2 # seconds
# -----

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=3)

print(f"Sensor client started → sending to {SERVER_IP}:{SERVER_PORT}")
```

```

GNU nano 7.2                                     sensor_client.py
time.sleep(2) # Wait for Arduino to initialize

while True:
    try:
        line = arduino.readline().decode("utf-8").strip()
        if not line:
            continue

        temp = float(line)

        # Binary format: zone_id (1 byte unsigned) + temperature (4 bytes float)
        packet = struct.pack("!Bf", ZONE_ID, temp)
        sock.sendto(packet, (SERVER_IP, SERVER_PORT))

        print(f"Sent → Zone {ZONE_ID}: {temp:.2f} °C")

    except ValueError:
        print("Bad reading from Arduino, skipping...")
    except Exception as e:
        print(f"Error: {e}")

    time.sleep(SEND_INTERVAL)

```

A.3 Server (server.py)

```

GNU nano 7.2                                     sever.py *
CSV_FILE = 'temperature_log.csv'

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((SERVER_IP, SERVER_PORT))
alert_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
zone_readings = {}

with open(CSV_FILE, 'w', newline='') as f:
    csv.writer(f).writerow(['Timestamp', 'Zone', 'Temp', 'ZoneAvg', 'OverallAvg', 'Alert'])

while True:
    try:
        data, addr = sock.recvfrom(1024)
        zone_id, temp = struct.unpack('!Bf', data)
        timestamp = datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        zone_readings.setdefault(zone_id, []).append(temp)
        zone_avg = sum(zone_readings[zone_id]) / len(zone_readings[zone_id])
        all_temps = [t for r in zone_readings.values() for t in r]
        overall_avg = sum(all_temps) / len(all_temps)
        alert = temp > THRESHOLD
        if alert:
            msg = f'ALERT! Zone {zone_id}: {temp:.2f}C exceeds {THRESHOLD}C!'
            alert_sock.sendto(msg.encode(), (ALERT_CLIENT_IP, ALERT_PORT))
    except:
        pass

```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark
 ^X Exit ^R Read File ^U Replace ^J Paste ^I Justify ^V Go To Line M-E Redo M-G Copy

```

    if alert:
        msg = f'ALERT! Zone {zone_id}: {temp:.2f}C exceeds {THRESHOLD}C!'
        alert_sock.sendto(msg.encode(), (ALERT_CLIENT_IP, ALERT_PORT))
    with open(CSV_FILE, 'a', newline='') as f:
        csv.writer(f).writerow([timestamp, zone_id, f'{temp:.2f}',
                                f'{zone_avg:.2f}', f'{overall_avg:.2f}', 'YES' if alert else 'NO'])
    print(f'[{timestamp}] Zone {zone_id}: {temp:.2f}°C')
except struct.error:
    print('Malformed packet, skipping...')
except Exception as e:
    print(f'Error: {e}')

```

^G Help
^O Write Out
^W Where Is
^K Cut
^T Execute
^C Location

A.4 Alert Client (alert_client.py)

GNU nano 7.2

alert_client.py

```

import socket

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(('0.0.0.0', 5006))
print('Alert client listening on port 5006...')

while True:
    data, addr = sock.recvfrom(1024)
    print(f'*** {data.decode()} ***')

```

B. Wiring Diagram

